

LAB 10. HISTOGRAM of ORIENTED GRADIENTS (HOG) & IT'S APPLICATION

Computer Vision

2025-2026

Mục lục

1	Thông tin chung của bài thực hành	4
1.1	Kiến thức tiên quyết	4
1.2	Chuẩn đầu ra	4
2	HOG trong bức tranh chung của môn học	5
2.1	Liên hệ với những gì đã học	5
3	Mục tiêu học tập chi tiết và kế hoạch buổi lab	6
3.1	Mục tiêu theo mức độ	6
4	Cơ sở lý thuyết cần nhắc lại	6
4.1	Gradient ảnh	6
4.2	Ý nghĩa của magnitude và orientation	7
4.3	Vì sao dùng khoảng góc $[0, 180)$?	7
4.4	Histogram hướng	7
4.5	Chuẩn hóa cục bộ	7
5	Pipeline tổng quát của HOG	7
5.1	Mô tả bằng lời	8
5.2	Sơ đồ pipeline	8
5.3	Ý nghĩa của cell và block	9
6	Thiết kế ảnh minh họa và dữ liệu thử nghiệm	9
7	Cài đặt HOG từng bước bằng Numpy	9

7.1	Bước 1: Đọc ảnh và tiền xử lý	10
7.2	Bước 2: Tính gradient ảnh	10
7.3	Bước 3: Lượng tử hóa hướng gradient	11
7.4	Bước 4: Tạo histogram cho từng cell	12
7.5	Bước 5: Chuẩn hóa theo block	13
7.6	Bước 6: Tạo vector đặc trưng HOG cuối cùng	14
8	Thực quan hóa HOG	15
8.1	Gợi ý code hiển thị cơ bản	15
8.2	Hình minh họa khái niệm	16
8.3	Nhận xét kết quả kỳ vọng	16
8.4	Bài tập nâng cao	16
9	Dùng thư viện có sẵn để trích xuất HOG	17
9.1	Ví dụ với scikit-image	17
9.2	Nội dung cần so sánh với bản tự cài đặt	17
10	Ứng dụng 1: Phân loại ảnh bằng HOG	17
10.1	Bài toán gợi ý	17
10.2	Pipeline	18
10.3	Code minh họa	18
10.4	Giải thích từng bước	19
10.5	Nhận xét kết quả kỳ vọng	20
10.6	Gợi ý hiển thị kết quả side-by-side	20
10.7	Bài tập nâng cao cho ứng dụng phân loại	20
11	Ứng dụng 2: Phát hiện đối tượng cơ bản bằng HOG	20
11.1	Ý tưởng tổng quát	20
11.2	Sơ đồ detector cổ điển	21
11.3	Chuẩn bị dữ liệu positive/negative	21
11.4	Sliding window	21
11.5	Image pyramid	22
11.6	Huấn luyện bộ phân loại patch	23

11.7 Quét phát hiện trên ảnh lớn	23
11.8 Non-Maximum Suppression (NMS)	24
11.9 Minh họa NMS	25
11.10 Nhận xét kết quả kỳ vọng	25
11.11 Bài tập nâng cao cho detector	25
12 Phần so sánh và phân tích kết quả	26
12.1 So sánh thủ công và thư viện có sẵn	26
12.2 Các hình minh họa nên có trong báo cáo của sinh viên	26
12.3 Ví dụ khung đánh giá kết quả	27
13 Bài tập theo từng phần	27
13.1 Sau phần xây dựng HOG	27
13.2 Sau phần phân loại ảnh	27
13.3 Sau phần detector	27
14 Bài tập tổng hợp cuối buổi	27
15 Hướng dẫn viết báo cáo thực hành	28
16 Những nhận xét sự phạm cần nhấn mạnh	29
17 Phụ lục A: Gợi ý cấu trúc notebook đi kèm	29
18 Kết luận	29

1. Thông tin chung của bài thực hành

Tên bài: Đặc trưng Histogram of Oriented Gradients (HOG) và ứng dụng trong nhận dạng ảnh, phát hiện đối tượng.

Mức độ: Cơ bản đến trung bình, định hướng thuật toán cổ điển.

Ngôn ngữ triển khai: Python, Numpy, OpenCV, scikit-image, scikit-learn.

Ý tưởng sơ phạm chính của bài lab. Bài thực hành này được thiết kế như một cầu nối giữa các nội dung xử lý ảnh cơ sở mà sinh viên đã học (gradient, tích chập, histogram, lọc, dò biên, phân đoạn) và các pipeline nhận dạng cổ điển trong thị giác máy tính. Thay vì gọi trực tiếp hàm HOG từ thư viện, sinh viên sẽ tự xây dựng HOG từng bước, nhìn được ý nghĩa của từng phép toán, rồi sau đó mới chuyển sang ứng dụng phân loại và phát hiện đối tượng.

1.1. Kiến thức tiên quyết

Sinh viên cần đảm bảo đã nắm được các kiến thức về:

- Ảnh xám và ảnh màu, hệ tọa độ ảnh, lấy mẫu và lượng tử hóa.
- Các toán tử tăng cường ảnh cơ bản, chuẩn hóa độ sáng, biến đổi histogram.
- Tích chập, bộ lọc trung bình, Gaussian, Sobel, Laplacian.
- Gradient ảnh, dò biên, vai trò của magnitude và orientation.
- Các khái niệm đặc trưng ảnh, đặc trưng cục bộ, đặc trưng toàn cục.
- Kiến thức lý thuyết cơ bản về HOG, nhưng chưa nhất thiết tự cài đặt pipeline đầy đủ.

1.2. Chuẩn đầu ra

Sau bài thực hành này, sinh viên cần có khả năng:

1. Giải thích được vì sao HOG dựa trên gradient thay vì cường độ mức xám trực tiếp.
2. Mô tả rõ từng bước: gradient $\rightarrow$ histogram cell $\rightarrow$ block normalization $\rightarrow$ vector đặc trưng cuối.
3. Tự viết được một phiên bản HOG cơ bản bằng Numpy.
4. Sử dụng thư viện có sẵn để trích xuất HOG và kiểm chứng kết quả.
5. Xây dựng được một pipeline phân loại ảnh bằng HOG.

6. Hiểu và cài đặt được pipeline detector cổ điển: sliding window, image pyramid, score threshold, NMS.
7. Phân tích được trường hợp nào HOG phù hợp, trường hợp nào HOG kém hiệu quả.

2. HOG trong bức tranh chung của môn học

Trong các bài trước, sinh viên chủ yếu làm việc ở mức xử lý ảnh: tăng cường chất lượng ảnh, trích xuất biên, phân đoạn vùng, thao tác hình thái hoặc biến đổi miền tần số. Những công cụ đó trả lời các câu hỏi kiểu: *ảnh có biên ở đâu, vùng nào sáng/tối, vùng nào thuộc đối tượng, vùng nào là nền*. Khi chuyển sang các bài toán thị giác máy tính cổ điển, mục tiêu thay đổi: ta muốn một **biểu diễn đặc trưng** ngắn gọn nhưng đủ giàu thông tin để phục vụ nhận dạng.

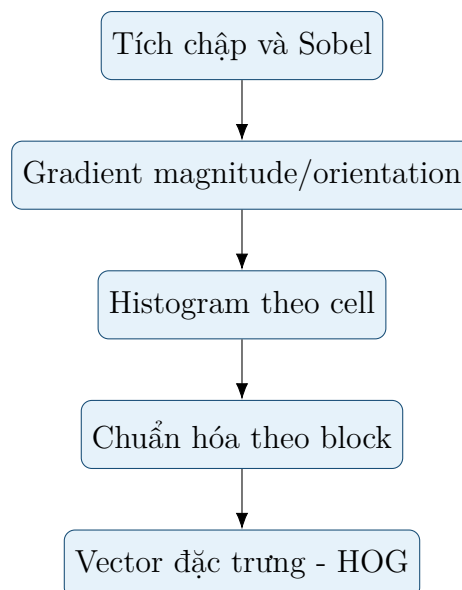
HOG là một ví dụ điển hình. Ta không dùng trực tiếp ma trận điểm ảnh gốc vì:

- số chiều quá lớn,
- nhạy với thay đổi sáng/tối,
- khó khái quát cho nhận dạng hình dạng.

Thay vào đó, HOG sử dụng **phân bố hướng gradient cục bộ**. Nói cách khác, HOG bỏ qua nhiều chi tiết texture không cần thiết và tập trung vào **cấu trúc cạnh và hình dáng**. Đây là lý do HOG từng rất nổi bật trong các hệ thống nhận dạng cổ điển như phát hiện người đi bộ, nhận dạng biển báo, chữ số, công cụ, vật thể trên nền tương đối đơn giản.

2.1. Liên hệ với những gì đã học

Kết nối lại các khối kiến thức đã học:



Vì vậy không nên xem HOG là một “thuật toán mới hoàn toàn”, mà là sự ghép nối hợp lý của nhiều khái niệm đã học trước đó.

3. Mục tiêu học tập chi tiết và kế hoạch buổi lab

3.1. Mục tiêu theo mức độ

Mức hiểu thuật toán:

- hiểu vai trò của gradient,
- hiểu vì sao cần cell và block,
- hiểu tại sao block normalization làm HOG ổn định hơn trước thay đổi sáng/tương phản.

Mức lập trình:

- tự cài đặt HOG cơ bản bằng Numpy,
- dùng OpenCV/scikit-image để kiểm tra chéo,
- xây dựng pipeline phân loại và phát hiện.

Mức phân tích:

- so sánh nhiều cấu hình HOG,
- nhận xét dữ liệu nào hợp với HOG,
- đánh giá sai số và nguyên nhân false positive/false negative.

4. Cơ sở lý thuyết cần nhắc lại

4.1. Gradient ảnh

Cho ảnh xám $I(x, y)$, gradient theo hai trục được xấp xỉ bằng các bộ lọc đạo hàm như Sobel:

$$G_x = I * S_x, \quad G_y = I * S_y \quad (1)$$

trong đó

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (2)$$

Độ lớn gradient:

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2} \quad (3)$$

Hướng gradient:

$$\theta(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)). \quad (4)$$

4.2. Ý nghĩa của magnitude và orientation

Magnitude cho biết biên mạnh tới mức nào; orientation cho biết biên đổi theo phương nào. Với nhận dạng hình dạng, thông tin quan trọng thường nằm nhiều hơn ở **hướng** và **sự phân bố cục bộ của hướng**, thay vì giá trị sáng tuyệt đối. Đây chính là trực giác cốt lõi của HOG.

4.3. Vì sao dùng khoảng góc $[0, 180)$?

Trong nhiều cài đặt HOG cổ điển, gradient không hướng tuyệt đối theo chiều mũi tên mà chỉ quan tâm đến *phương cạnh*. Khi đó góc 0° và 180° được coi là tương đương. Đây là unsigned orientation. Với nhiều bài toán thị giác, cách này giúp đặc trưng gọn hơn và bền hơn.

4.4. Histogram hướng

Nếu chia khoảng $[0, 180)$ thành B bin thì độ rộng mỗi bin là:

$$\Delta = \frac{180}{B}. \quad (5)$$

Với mỗi pixel, ta bỏ phiếu vào bin tương ứng bằng trọng số magnitude. Phiên bản tốt hơn là nội suy sang hai bin kề nhau để giảm răng cưa và làm histogram mượt hơn.

4.5. Chuẩn hóa cục bộ

Nếu ánh sáng toàn ảnh hoặc tương phản cục bộ thay đổi, biên độ gradient thay đổi theo. Do đó chỉ dùng histogram thô sẽ không ổn định. HOG giải quyết bằng cách ghép vài cell lân cận thành block rồi chuẩn hóa:

$$\hat{v} = \frac{v}{\sqrt{\|v\|_2^2 + \varepsilon^2}} \quad (6)$$

trong đó v là vector histogram block, ε nhỏ để tránh chia cho 0.

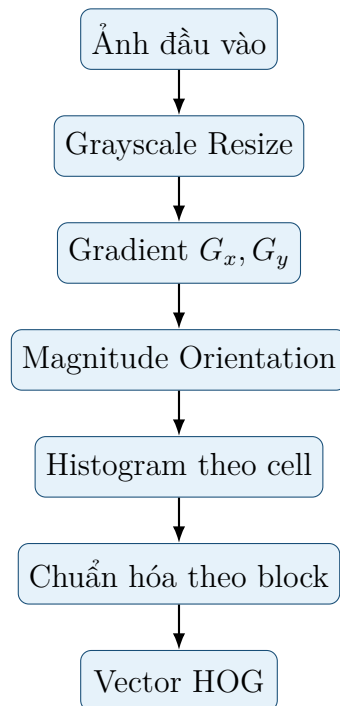
5. Pipeline tổng quát của HOG

5.1. Mô tả bằng lời

Với một ảnh đầu vào, pipeline HOG cơ bản gồm các bước:

1. chuẩn hóa kích thước ảnh,
2. chuyển sang ảnh xám nếu cần,
3. tính gradient G_x, G_y ,
4. suy ra magnitude và orientation,
5. chia ảnh thành các cell nhỏ,
6. xây dựng histogram hướng cho từng cell,
7. gom nhiều cell thành các block chồng lấn,
8. chuẩn hóa từng block,
9. nối tất cả các block để được vector đặc trưng cuối cùng.

5.2. Sơ đồ pipeline



Hình 1: Pipeline tổng quát của đặc trưng HOG.

5.3. Ý nghĩa của cell và block

Cell giúp giữ lại thông tin cục bộ về phân bố hướng cạnh. Nếu histogram được gom trên toàn ảnh, ta mất hoàn toàn thông tin vị trí. **Block** lại giúp chuẩn hóa cục bộ, nhờ đó đặc trưng ít phụ thuộc hơn vào điều kiện sáng.

6. Thiết kế ảnh minh họa và dữ liệu thử nghiệm

Để sinh viên thực sự thấy được HOG hoạt động tốt ra sao, nên chọn các loại ảnh có đặc điểm sau:

- đối tượng có biên rõ,
- hình dáng đủ khác nhau giữa các lớp,
- nền tương đối đơn giản ở phiên bản đầu,
- kích thước đối tượng không quá nhỏ so với ảnh.

Ví dụ tốt:

- chai nước, cốc, kéo, bút,
- chữ số viết tay hoặc ký tự in đậm,
- biển báo giao thông có hình học đặc trưng,
- một số công cụ có silhouette rõ.

Ví dụ không quá phù hợp ở giai đoạn đầu:

- cảnh thiên nhiên phức tạp,
- dữ liệu mà khác biệt chính nằm ở màu sắc chứ không phải hình dạng,
- vật thể quá nhỏ trong ảnh rộng,
- nhiều che khuất, biến dạng hoặc quay mạnh.

7. Cài đặt HOG từng bước bằng Numpy

Trong phần này, mục tiêu không phải là viết một bản tối ưu hóa cao, mà là viết một bản **chi tiết các bước thuật toán**, để các bạn nhìn được từng bước dữ liệu biến đổi như thế nào.

7.1. Bước 1: Đọc ảnh và tiền xử lý

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("sample.png", cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (128, 128))

plt.imshow(img, cmap="gray")
plt.title("Input grayscale image")
plt.axis("off")
plt.show()
```

Nhận xét kết quả kỳ vọng

Ảnh sau khi resize nên vẫn giữ được cấu trúc hình dáng chính. Nếu resize quá nhỏ, nhiều cạnh quan trọng sẽ bị triệt tiêu; nếu resize không đồng nhất giữa các mẫu, vector đặc trưng sẽ khó so sánh trực tiếp.

Hướng dẫn hiển thị side-by-side

Các bạn hãy tự viết code để hiển thị song song:

- ảnh gốc,
- ảnh xám,
- ảnh sau resize,

để sinh viên thấy rằng bước chuẩn hóa kích thước không chỉ mang tính kỹ thuật mà ảnh hưởng trực tiếp đến cấu trúc đặc trưng.

7.2. Bước 2: Tính gradient ảnh

```
Gx = cv2.Sobel(img.astype(np.float32), cv2.CV_32F, 1, 0, ksize=3)
Gy = cv2.Sobel(img.astype(np.float32), cv2.CV_32F, 0, 1, ksize=3)

magnitude = np.sqrt(Gx**2 + Gy**2)
orientation = np.rad2deg(np.arctan2(Gy, Gx)) % 180
```

```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].imshow(Gx, cmap="gray")
ax[0].set_title("Gradient Gx")
ax[1].imshow(Gy, cmap="gray")
ax[1].set_title("Gradient Gy")
ax[2].imshow(magnitude, cmap="gray")
ax[2].set_title("Gradient magnitude")
for a in ax:
    a.axis("off")
plt.show()
```

Diễn giải

G_x làm nổi bật các cạnh gần thẳng đứng; G_y làm nổi bật các cạnh gần nằm ngang. Magnitude làm lộ ra những nơi chuyển mức xám mạnh nhất, còn orientation cho biết hướng thay đổi tương ứng. Đây là nơi tri thức về Sobel, gradient và dò biên từ các bài trước xuất hiện rất rõ.

Nhận xét kết quả kỳ vọng

Ở ảnh có hình dáng rõ như chai, cốc, người đứng hoặc chữ số, magnitude map sẽ thể hiện rất rõ đường bao chính. Với ảnh nhiều texture ngẫu nhiên, magnitude map sẽ rối hơn và HOG cũng khó tạo ra mô tả ổn định hơn.

7.2.1. Bài tập nâng cao

1. Thay Sobel bằng Prewitt hoặc đạo hàm sai phân đơn giản và so sánh.
2. Làm trơn Gaussian trước khi tính gradient, đánh giá độ mượt của orientation map.
3. So sánh gradient nếu dùng ảnh xám so với khi chỉ lấy riêng một kênh màu.

7.3. Bước 3: Lượng tử hóa hướng gradient

Phiên bản đơn giản nhất: mỗi pixel bỏ phiếu toàn bộ magnitude vào một bin gần nhất. Phiên bản tốt hơn: magnitude được nội suy tuyến tính vào hai bin kề nhau.

```
def vote_to_bins(angle, mag, num_bins=9):
    bin_width = 180 / num_bins
    pos = angle / bin_width
    left = int(np.floor(pos)) % num_bins
```

```

right = (left + 1) % num_bins
frac = pos - np.floor(pos)
return left, right, mag * (1 - frac), mag * frac

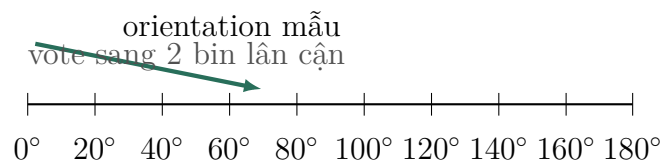
```

Diễn giải

Nội suy theo hai bin giúp histogram mượt hơn, ít phụ thuộc vào ranh giới cứng giữa các bin. Ví dụ, nếu một pixel có orientation gần đúng giữa hai bin, việc dồn toàn bộ phiếu vào một bin sẽ tạo sai lệch nhân tạo.

Gợi ý minh họa

Nên chèn một hình quạt tròn hoặc một thanh chia 9 bin trên khoảng 0 đến 180 độ, sau đó vẽ một góc mẫu rơi giữa hai bin để giải thích cơ chế voting.



Hình 2: Minh họa việc lượng tử hóa orientation vào các bin histogram.

7.4. Bước 4: Tạo histogram cho từng cell

```

def compute_cell_histograms(magnitude, orientation, cell_size=8,
    num_bins=9):
    h, w = magnitude.shape
    n_cells_y = h // cell_size
    n_cells_x = w // cell_size
    hist = np.zeros((n_cells_y, n_cells_x, num_bins), dtype=np.float32)

    for cy in range(n_cells_y):
        for cx in range(n_cells_x):
            for iy in range(cell_size):
                for ix in range(cell_size):
                    y = cy * cell_size + iy
                    x = cx * cell_size + ix
                    angle = orientation[y, x]
                    mag = magnitude[y, x]
                    left, right, v1, v2 = vote_to_bins(angle, mag,
                        num_bins)
                    hist[cy, cx, left] += v1

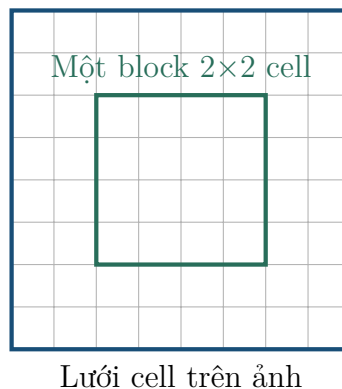
```

```
hist[cy, cx, right] += v2
return hist
```

Cell có ý nghĩa gì?

Mỗi cell là một vùng nhỏ của ảnh, thường là 8×8 pixel. Histogram của cell trả lời câu hỏi: *trong vùng nhỏ này, các cạnh chủ yếu hướng theo phương nào và mạnh đến đâu?* Nhờ chia thành nhiều cell, HOG vừa giữ được tính cục bộ, vừa không quá nhạy như việc xét từng pixel riêng lẻ.

Minh họa cell và block



Hình 3: Minh họa chia ảnh thành các cell và một block chồng lấn gồm 2×2 cell.

Nhận xét kết quả kỳ vọng

Nếu ảnh có đối tượng với đường bao rõ, các cell nằm gần biên sẽ có histogram nổi bật theo một vài hướng chiếm ưu thế. Các cell nằm trong vùng gần đồng nhất sẽ có histogram yếu hơn nhiều.

7.4.1. Bài tập nâng cao

1. Thử cell size bằng 4, 8, 16 và so sánh tính cục bộ của đặc trưng.
2. Phân tích vì sao cell quá nhỏ dễ nhạy nhiễu, cell quá lớn dễ mất chi tiết vị trí.
3. Thử bỏ qua các pixel có magnitude quá nhỏ để giảm nhiễu trong histogram.

7.5. Bước 5: Chuẩn hóa theo block

```
def block_normalize(cell_hist, block_size=2, eps=1e-6):
    n_cells_y, n_cells_x, num_bins = cell_hist.shape
    blocks = []
    for y in range(n_cells_y - block_size + 1):
        for x in range(n_cells_x - block_size + 1):
            block = cell_hist[y:y+block_size, x:x+block_size, :].ravel()
            block = block / np.sqrt(np.sum(block**2) + eps**2)
            blocks.append(block)
    return np.array(blocks, dtype=np.float32)
```

Diễn giải

Block normalization là bước rất quan trọng. Nếu hai ảnh có cùng hình dạng nhưng điều kiện sáng khác nhau, histogram cell có thể thay đổi theo hệ số. Khi chuẩn hóa block, ta giảm ảnh hưởng đó và nhấn mạnh *tương quan tương đối* giữa các hướng thay vì biên độ tuyệt đối.

Nhận xét kết quả kỳ vọng

Nếu bỏ bước chuẩn hóa, độ chính xác của bộ phân loại thường giảm rõ khi ảnh có khác biệt chiếu sáng hoặc tương phản. Đây là thí nghiệm rất nên yêu cầu sinh viên tự kiểm chứng.

7.5.1. Bài tập nâng cao

1. So sánh L1, L2, L2-Hys.
2. Tắt hẳn block normalization rồi đánh giá lại mô hình phân loại.

7.6. Bước 6: Tạo vector đặc trưng HOG cuối cùng

```
def hog_feature_manual(img, cell_size=8, block_size=2, num_bins=9):
    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    img = img.astype(np.float32)
    Gx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
    Gy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
    magnitude = np.sqrt(Gx**2 + Gy**2)
    orientation = np.rad2deg(np.arctan2(Gy, Gx)) % 180
```

```
cell_hist = compute_cell_histograms(magnitude, orientation,
    cell_size, num_bins)
blocks = block_normalize(cell_hist, block_size)
return blocks.ravel()
```

Giả sử ảnh có kích thước $H \times W$, cell size là c , block size là b (tính theo số cell), và số orientation bins là B . Khi đó:

$$n_y = H/c, \quad n_x = W/c, \quad (7)$$

$$n_y^{\text{block}} = n_y - b + 1, \quad n_x^{\text{block}} = n_x - b + 1, \quad (8)$$

chiều dài vector đặc trưng là:

$$(n_y - b + 1)(n_x - b + 1)(b^2 B). \quad (9)$$

7.6.1. Ví dụ

Với ảnh 64×128 , cell 8×8 , block 2×2 , $B = 9$:

$$n_y = 128/8 = 16, \quad (10)$$

$$n_x = 64/8 = 8, \quad (11)$$

$$n_y^{\text{block}} = 15, \quad (12)$$

$$n_x^{\text{block}} = 7, \quad (13)$$

$$d = 15 \cdot 7 \cdot 4 \cdot 9 = 3780. \quad (14)$$

Đây là kích thước rất nổi tiếng trong pipeline HOG cho cửa sổ phát hiện người đi bộ.

8. Trực quan hóa HOG

Trực quan hóa là phần rất quan trọng vì nó giúp sinh viên hiểu HOG *nhìn thấy gì*. Với mỗi cell, có thể vẽ các đoạn thẳng ngắn tại tâm cell theo từng hướng bin, độ dài tỉ lệ với giá trị histogram tương ứng.

8.1. Gợi ý code hiển thị cơ bản

Có thể dùng `skimage.feature.hog(..., visualize=True)` để lấy ảnh visualization, hoặc tự vẽ bằng Matplotlib/NumPy.

```
from skimage.feature import hog
from skimage import exposure
```

```
fd, hog_image = hog(
    img,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)

hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0,
    10))
```

8.2. Hình minh họa khái niệm



Minh họa các đoạn định hướng biểu diễn histogram của từng cell

Hình 4: Sơ đồ khái niệm của HOG visualization. Trong thực tế, độ dài từng đoạn tỉ lệ với giá trị histogram ở hướng tương ứng.

8.3. Nhận xét kết quả kỳ vọng

Với ảnh có cấu trúc hình dáng rõ, HOG visualization sẽ bộc lộ khá tốt đường bao chính của đối tượng. Tuy nhiên, không nên hiểu ảnh visualization là ảnh biên thông thường; nó là biểu diễn tổng hợp của phân bố hướng trong từng cell.

8.4. Bài tập nâng cao

1. So sánh HOG visualization của ảnh gốc và ảnh blur.
2. So sánh ảnh gốc và ảnh quay 15° , 30° ; nhận xét tính nhạy quay của HOG.
3. Tự viết hàm visualization đơn giản dựa trên cell histogram đã tính được.

9. Dùng thư viện có sẵn để trích xuất HOG

Sau khi đã hiểu bản chất thuật toán, sinh viên nên dùng thư viện có sẵn để tiết kiệm thời gian khi xây dựng ứng dụng đầy đủ.

9.1. Ví dụ với scikit-image

```
from skimage.feature import hog

feat = hog(
    img,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=False,
    block_norm='L2-Hys'
)
```

9.2. Nội dung cần so sánh với bản tự cài đặt

- chiều dài vector đặc trưng,
- thời gian chạy,
- chất lượng khi đưa vào classifier,
- sự khác biệt do nội suy orientation hoặc chuẩn hóa block.

Thông điệp cần nhấn mạnh. Tự cài đặt giúp hiểu thuật toán; thư viện có sẵn giúp triển khai ứng dụng tin cậy và nhanh hơn. Một lab tốt nên cho sinh viên trải nghiệm cả hai phía này.

10. Ứng dụng 1: Phân loại ảnh bằng HOG

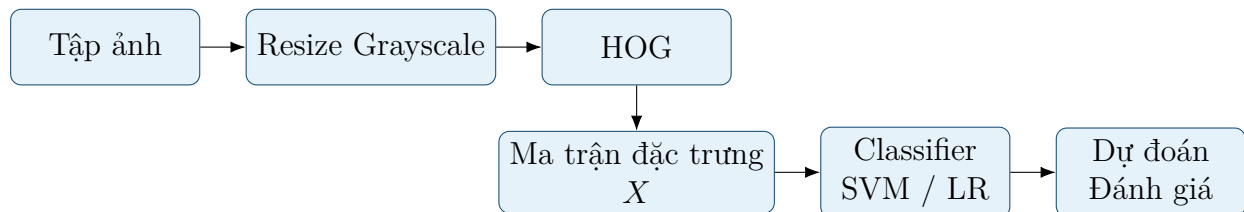
Đây là ứng dụng nên triển khai đầu tiên vì dễ theo dõi hơn phát hiện đối tượng. Ở đây, ảnh đầu vào thường chỉ chứa một đối tượng chính hoặc một lớp hình dạng khá nổi bật.

10.1. Bài toán gợi ý

- phân loại chai / cốc / kéo,

- phân loại chữ số,
- phân loại một số biển báo đơn giản,
- phân loại biểu tượng đen-trắng theo hình dạng.

10.2. Pipeline



Hình 5: Pipeline phân loại ảnh bằng đặc trưng HOG.

10.3. Code minh họa

```
import os
import cv2
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix,
    accuracy_score
from skimage.feature import hog

def extract_hog_skimage(gray):
    feat = hog(
        gray,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        visualize=False,
        block_norm='L2-Hys'
    )
    return feat

X_features = []
y_labels = []
root_dir = 'dataset'
classes = sorted(os.listdir(root_dir))
```

```
for cls in classes:
    cls_dir = os.path.join(root_dir, cls)
    for fname in os.listdir(cls_dir):
        path = os.path.join(cls_dir, fname)
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            continue
        img = cv2.resize(img, (128, 128))
        feat = extract_hog_skimage(img)
        X_features.append(feat)
        y_labels.append(cls)

X_features = np.array(X_features)
y_labels = np.array(y_labels)

X_train, X_test, y_train, y_test = train_test_split(
    X_features, y_labels, test_size=0.2, random_state=42, stratify=
    y_labels
)

clf = LinearSVC()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

10.4. Giải thích từng bước

1. **Chuẩn hóa kích thước ảnh** giúp vector HOG giữa các mẫu có cùng chiều dài.
2. **Trích xuất HOG** chuyển mỗi ảnh sang một vector đặc trưng có ý nghĩa hình dáng.
3. **Tạo ma trận X** với mỗi hàng là một ảnh.
4. **Huấn luyện Linear SVM** là một lựa chọn kinh điển, thường hoạt động khá tốt với đặc trưng HOG.
5. **Đánh giá** nên dùng accuracy, confusion matrix và phân tích lỗi theo từng lớp.

10.5. Nhận xét kết quả kỳ vọng

Nếu các lớp có khác biệt hình dạng rõ, HOG + Linear SVM thường cho kết quả khá tốt ngay cả với tập dữ liệu cỡ vừa. Tuy nhiên, nếu các lớp khác nhau chủ yếu về màu hoặc texture, HOG có thể kém hơn các đặc trưng khác.

10.6. Gợi ý hiển thị kết quả side-by-side

Nên hiển thị:

- vài ảnh đúng dự đoán,
- vài ảnh sai dự đoán,
- confusion matrix,
- HOG visualization của một số mẫu tiêu biểu.

10.7. Bài tập nâng cao cho ứng dụng phân loại

1. So sánh Linear SVM, Logistic Regression, KNN.
2. So sánh HOG với histogram mức xám hoặc vector ảnh resize trực tiếp.
3. Thử PCA sau HOG rồi mới phân loại.
4. Đánh giá ảnh hưởng của dữ liệu bị xoay nhẹ hoặc blur nhẹ.

11. Ứng dụng 2: Phát hiện đối tượng cơ bản bằng HOG

Phần này giúp sinh viên tiếp cận một detector cổ điển trước thời kỳ deep learning: quét nhiều cửa sổ ứng viên, chấm điểm từng cửa sổ, rồi loại bớt hộp trùng lặp bằng NMS.

11.1. Ý tưởng tổng quát

Ta huấn luyện một bộ phân loại nhị phân trên các patch kích thước chuẩn:

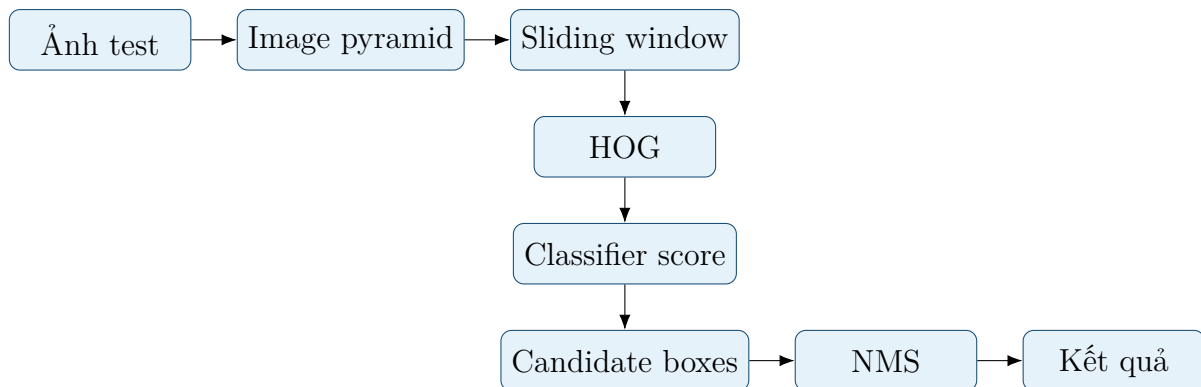
- **positive patches**: chứa đối tượng,
- **negative patches**: không chứa đối tượng.

Khi chạy trên ảnh lớn:

1. tạo nhiều tỷ lệ bằng image pyramid,

2. quét sliding window,
3. tính HOG cho mỗi cửa sổ,
4. dùng classifier chấm điểm,
5. giữ các candidate boxes có score vượt ngưỡng,
6. dùng NMS để chỉ còn các phát hiện mạnh nhất.

11.2. Sơ đồ detector cổ điển



Hình 6: Pipeline phát hiện đối tượng cơ bản với HOG.

11.3. Chuẩn bị dữ liệu positive/negative

Đối với một bài lab cơ bản, nên chọn một vật thể có hình dáng tương đối ổn định như chai nước, cốc, kéo hoặc một vật dụng cố định trong phòng. Positive patches được cắt chứa vật thể; negative patches lấy từ nền, mặt bàn, tường, hoặc các vùng không chứa vật thể đó.

11.4. Sliding window

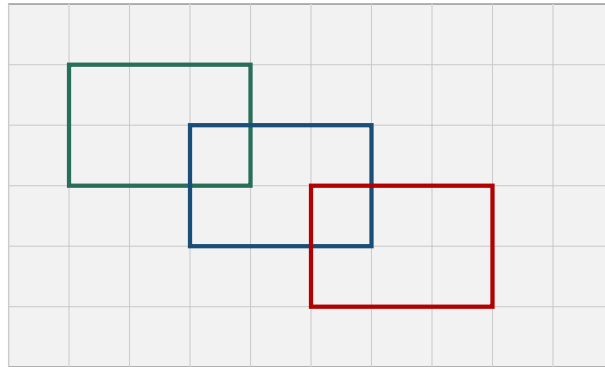
```

def sliding_window(image, step_size, window_size):
    h, w = image.shape[:2]
    win_w, win_h = window_size
    for y in range(0, h - win_h + 1, step_size):
        for x in range(0, w - win_w + 1, step_size):
            yield (x, y, image[y:y+win_h, x:x+win_w])
  
```

11.4.1. Giải thích

Sliding window là thao tác quét một cửa sổ kích thước cố định trên toàn ảnh. Bước nhảy nhỏ giúp phủ dày hơn nhưng chậm hơn; bước nhảy lớn thì nhanh hơn nhưng dễ bỏ sót vật thể.

11.4.2. Minh họa sliding window



Các vị trí của sổ trượt minh họa trên ảnh

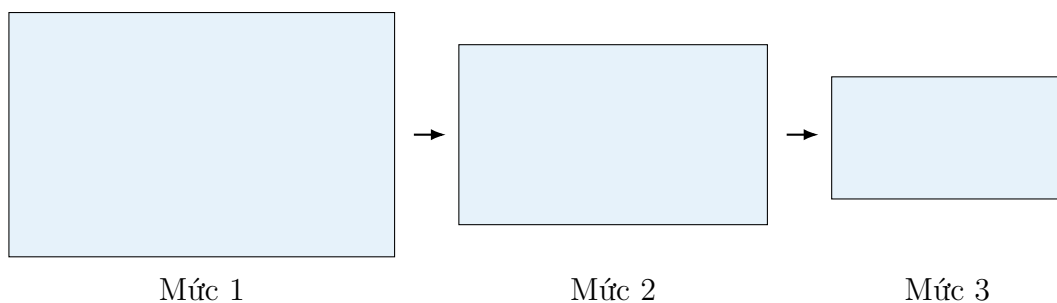
Hình 7: Minh họa ý tưởng sliding window.

11.5. Image pyramid

```
def image_pyramid(image, scale=1.25, min_size=(64, 64)):
    yield image
    while True:
        w = int(image.shape[1] / scale)
        h = int(image.shape[0] / scale)
        image = cv2.resize(image, (w, h))
        if w < min_size[0] or h < min_size[1]:
            break
        yield image
```

11.5.1. Giải thích

Một cửa sổ cố định không thể bao phủ tốt vật thể ở nhiều kích thước. Image pyramid xử lý điều này bằng cách co ảnh dần. Khi ảnh co lại, một vật thể lớn trong ảnh gốc có thể trở nên phù hợp với cửa sổ chuẩn.



Hình 8: Minh họa image pyramid: ảnh được thu nhỏ dần để quét vật thể ở nhiều tỷ lệ.

11.6. Huấn luyện bộ phân loại patch

```
from sklearn.svm import LinearSVC
from skimage.feature import hog

def extract_patch_hog(patch, out_size=(64, 64)):
    gray = cv2.cvtColor(patch, cv2.COLOR_BGR2GRAY) if len(patch.shape)
        == 3 else patch
    gray = cv2.resize(gray, out_size)
    feat = hog(
        gray,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        visualize=False,
        block_norm='L2-Hys'
    )
    return feat
```

11.7. Quét phát hiện trên ảnh lớn

```
def detect_objects(image, clf, window_size=(64, 64), step_size=8,
                    pyramid_scale=1.25, threshold=0.0):
    boxes = []
    scores = []

    for resized in image_pyramid(image, scale=pyramid_scale, min_size=
        window_size):
        scale_x = image.shape[1] / resized.shape[1]
        scale_y = image.shape[0] / resized.shape[0]

        for (x, y, window) in sliding_window(resized, step_size,
            window_size):
            feat = extract_patch_hog(window, out_size=window_size)
            score = clf.decision_function([feat])[0]
            if score > threshold:
                x1 = int(x * scale_x)
                y1 = int(y * scale_y)
                x2 = int((x + window_size[0]) * scale_x)
                y2 = int((y + window_size[1]) * scale_y)
                boxes.append([x1, y1, x2, y2])
```

```

        scores.append(score)

    return boxes, scores

```

11.8. Non-Maximum Suppression (NMS)

Vì nhiều cửa sổ gần nhau có thể cùng phát hiện một đối tượng, ta cần loại bỏ bớt các hộp trùng lặp. Độ đo chồng lấn phổ biến là IoU:

$$IoU(A, B) = \frac{\text{Area}(A \cap B)}{\text{Area}(A \cup B)}. \quad (15)$$

```

def compute_iou(boxA, boxB):
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])

    inter_w = max(0, xB - xA)
    inter_h = max(0, yB - yA)
    inter_area = inter_w * inter_h

    areaA = max(0, boxA[2] - boxA[0]) * max(0, boxA[3] - boxA[1])
    areaB = max(0, boxB[2] - boxB[0]) * max(0, boxB[3] - boxB[1])

    union = areaA + areaB - inter_area
    if union == 0:
        return 0.0
    return inter_area / union

def non_max_suppression(boxes, scores, iou_threshold=0.3):
    if len(boxes) == 0:
        return [], []

    boxes = np.array(boxes)
    scores = np.array(scores)
    idxs = np.argsort(scores)[::-1]
    keep = []

    while len(idxs) > 0:
        current = idxs[0]

```



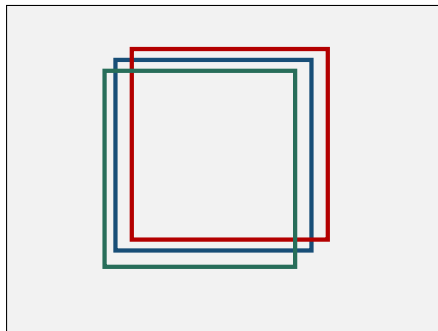
```
keep.append(current)
remaining = []

for idx in idxs[1:]:
    iou = compute_iou(boxes[current], boxes[idx])
    if iou < iou_threshold:
        remaining.append(idx)

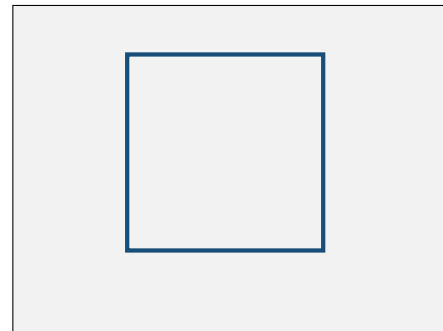
idxs = np.array(remaining)

return boxes[keep].tolist(), scores[keep].tolist()
```

11.9. Minh họa NMS



Trước NMS



Sau NMS

Hình 9: Minh họa nguyên lý của Non-Maximum Suppression.

11.10. Nhận xét kết quả kỳ vọng

Ở bài toán phát hiện cơ bản, sinh viên thường gặp hai loại lỗi chính:

- **false positive:** nền hoặc vật thể khác có hình dáng tương đối giống đối tượng mục tiêu,
- **false negative:** vật thể bị xoay, bị che khuất, quá nhỏ hoặc quá khác so với mẫu huấn luyện.

Điều quan trọng là sinh viên phải liên hệ lỗi quan sát được với bản chất của HOG, chứ không chỉ báo cáo số lượng hộp đúng/sai.

11.11. Bài tập nâng cao cho detector

1. Thử step size = 4, 8, 16 và đo thời gian chạy.

2. Thử pyramid scale = 1.1, 1.25, 1.5 và phân tích đánh đổi giữa tốc độ và độ bao phủ tỷ lệ.
3. So sánh NMS với các ngưỡng IoU khác nhau.
4. Cài đặt hard negative mining ở mức đơn giản.

12. Phần so sánh và phân tích kết quả

12.1. So sánh thủ công và thư viện có sẵn

Sinh viên nên được yêu cầu trình bày bảng so sánh:

Tiêu chí	HOG tự cài đặt bằng Numpy	HOG từ thư viện
Mức độ hiệu thuật toán	Rất tốt	Trung bình
Tốc độ chạy	Chậm hơn	Nhanh hơn
Độ linh hoạt sửa đổi pipeline	Cao	Phụ thuộc API
Khả năng dùng cho ứng dụng lớn	Hạn chế	Tốt hơn
Hỗ trợ hiểu	Rất cao	Cao khi dùng để kiểm chứng

12.2. Các hình minh họa nên có trong báo cáo của sinh viên

1. Ảnh gốc, ảnh gradient magnitude, HOG visualization.
2. Ảnh chia cell/block minh họa.
3. Confusion matrix cho bài toán phân loại.
4. Ví dụ đúng/sai của classifier.
5. Ảnh detector trước NMS và sau NMS.
6. Một số false positive và false negative tiêu biểu.

12.3. Ví dụ khung đánh giá kết quả

Cấu hình	Accuracy / mAP sơ bộ	Tốc độ	Nhận xét
HOG (9 bins, cell 8, block 2) + SVM	...	...	Cấu hình chuẩn
HOG (12 bins, cell 8, block 2) + SVM	...	...	Mô tả hướng chi tiết hơn
HOG (9 bins, cell 16, block 2) + SVM	...	...	Giảm tính cục bộ
No block normalization	...	...	Kiểm tra vai trò chuẩn hóa

13. Bài tập theo từng phần

13.1. Sau phần xây dựng HOG

1. So sánh gán cứng 1 bin với nội suy 2 bin.
2. So sánh unsigned orientation $[0, 180)$ với signed orientation $[0, 360)$.
3. Kết hợp ngưỡng magnitude để loại bỏ gradient yếu, phân tích ảnh hưởng lên vector HOG.

13.2. Sau phần phân loại ảnh

1. So sánh HOG với histogram mức xám và vector ảnh resize trực tiếp.
2. Đánh giá hiệu quả của PCA trước classifier.
3. Xây dựng một bảng lỗi theo từng lớp và giải thích bằng hình dạng đối tượng.

13.3. Sau phần detector

1. Phân tích chi phí tính toán khi thay step size và pyramid scale.
2. Thử thêm mirrored augmentation cho positive patches.
3. Cải tiến detector bằng hard negative mining đơn giản.
4. Đề xuất một chiến lược giảm số cửa sổ quét mà vẫn giữ chất lượng hợp lý.

14. Bài tập tổng hợp cuối buổi

Bài 1. Khảo sát tham số HOG

Trên một bộ dữ liệu phân loại đơn giản, hãy khảo sát ảnh hưởng của số bins, cell size, block size và chuẩn hóa block. Cần trình bày bảng kết quả và giải thích xu hướng.

Bài 2. So sánh đặc trưng

Dùng cùng một bộ dữ liệu, so sánh HOG với histogram mức xám và vector ảnh resize trực tiếp. Chọn cùng một classifier để bảo đảm công bằng.

Bài 3. Độ ổn định trước biến đổi hình học

Với cùng một ảnh đối tượng, tạo các phiên bản quay nhẹ, blur nhẹ, thay đổi sáng/tương phản. So sánh HOG visualization và độ thay đổi kết quả dự đoán.

Bài 4. Detector cơ bản

Tự thu thập dữ liệu positive/negative cho một vật thể đơn giản và xây dựng detector HOG + Linear SVM với sliding window + NMS. Trình bày kết quả trước và sau NMS trên ít nhất 5 ảnh test.

Bài 5. Đề xuất ứng dụng mở rộng

Đề xuất một bài toán thực tế có thể dùng HOG, mô tả dữ liệu, pipeline, các bước tiền xử lý và phân tích vì sao HOG phù hợp hoặc không phù hợp.

15. Hướng dẫn viết báo cáo thực hành

Sinh viên nên trình bày báo cáo theo cấu trúc:

1. Mục tiêu bài toán.
2. Mô tả dữ liệu và cách thu thập ảnh.
3. Cấu hình HOG sử dụng.
4. Bộ phân loại hoặc detector sử dụng.
5. Kết quả định lượng.
6. Kết quả trực quan.
7. Phân tích lỗi.
8. Ưu điểm, nhược điểm của HOG trong bối cảnh dữ liệu đã thử.
9. Hướng cải tiến tiếp theo.

16. Những nhận xét sơ phạm cần nhấn mạnh

- HOG là ví dụ rất tốt về việc xây dựng đặc trưng từ kiến thức xử lý ảnh cơ sở.
- Gradient, histogram và chuẩn hóa cục bộ không phải các chủ đề rời rạc; HOG cho thấy chúng kết hợp rất tự nhiên.
- HOG mạnh với hình dạng cục bộ, nhưng không bất biến hoàn toàn với quay, che khuất lớn, biến dạng mạnh hoặc thay đổi viewpoint lớn.
- Sliding window + NMS là tiền đề quan trọng để hiểu các detector hiện đại sau này.

17. Phụ lục A: Gợi ý cấu trúc notebook đi kèm

Sinh viên nên hoàn thiện tất cả bài tập theo các bước sau:

1. Imports và cấu hình hiển thị.
2. Đọc dữ liệu và tiền xử lý.
3. Tính gradient và trực quan hóa.
4. Tự cài đặt HOG bằng Numpy.
5. HOG visualization.
6. HOG từ thư viện có sẵn.
7. Ứng dụng phân loại ảnh.
8. Ứng dụng detector với sliding window.
9. NMS và hiển thị kết quả.
10. Bài tập mở rộng.

18. Kết luận

Bài lab này được thiết kế để sinh viên không chỉ biết gọi hàm HOG, mà còn hiểu cách một đặc trưng cổ điển được xây dựng từ những khái niệm rất cơ sở của xử lý ảnh: gradient, histogram và chuẩn hóa. Khi được nối sang các ứng dụng phân loại ảnh và phát hiện đối tượng, HOG trở thành một ví dụ rất đẹp về cách biến kiến thức xử lý ảnh thành một pipeline thị giác máy tính hoàn chỉnh.

Nếu triển khai đầy đủ theo tài liệu này, sinh viên sẽ có một nền tảng rất tốt để tiếp tục sang các chủ đề đặc trưng cục bộ, Bag of Visual Words, SIFT/SURF/ORB và sau đó là các detector học sâu hiện đại.